Question 1: Forward Propagation & Structure

Q: How do Recurrent Neural Networks (RNNs) handle sequential input data of varying lengths during forward propagation?

Solution :
RNNs handle sequential data of varying lengths by processing each sequence one time step at a time. The hidden state carries information from previous time steps, and the same weights are reused throughout the sequence. Therefore, the RNN can perform forward propagation for as many time steps as the sequence contains. When multiple sequences of different lengths are processed in a batch, padding and techniques such as masking or packed sequences can be used to handle the different lengths.

Question 2: Shared Parameters

Q: When an RNN is unrolled across multiple time steps, what happens to the weights and biases?

Solution :

When an RNN is unrolled across multiple time steps, the same weights and biases are shared across all time steps. The same parameters are reused to process each input and update the hidden state. During Backpropagation Through Time (BPTT), the gradients from all time steps are used to update these shared parameters.

Example:

Suppose the input sequence is:

“I love AI”

The RNN processes it as:

x_1 = “I” → RNN → h_1
x_2 = “love” → RNN → h_2
x_3 = “AI” → RNN → h_3

The same W_x, W□, and b are used at all three steps:

$h_1 = f(W_x x_1 + W \square h_0 + b)$

$h_2 = f(W_x x_2 + W \square h_1 + b)$

$h_3 = f(W_x x_3 + W \square h_2 + b)$

Therefore, the RNN does not create separate weights and biases for each word. It uses one shared set of parameters throughout the entire sequence.

Question 3: Exploding Gradient Problem

Q: During backpropagation over many time steps, what causes the gradient to "explode," and how does this affect training?

Solution :

The **exploding gradient problem** occurs in RNNs when gradients become extremely large during Backpropagation Through Time (BPTT). This mainly happens because the gradient is repeatedly multiplied by the same recurrent weight matrix across many time steps. If the relevant weights or eigenvalues have values greater than 1, these repeated multiplications can make the gradient grow exponentially.

Example:

Suppose the gradient is multiplied by **2** at every time step:

Step 1: 2
Step 2: 2 × 2 = 4
Step 3: 4 × 2 = 8
Step 4: 8 × 2 = 16
Step 5: 16 × 2 = 32

As the number of time steps increases, the gradient can become extremely large.

Effect on Training:

When gradients become very large, the weight updates also become very large. This can make training unstable, cause the loss to fluctuate or become extremely large, and in severe cases cause numerical overflow such as NaN values. As a result, the RNN may fail to learn properly.

A common technique used to reduce this problem is **gradient clipping**, where the gradient is limited to a predefined maximum value before updating the model parameters.

Question 4: Vanishing Gradient Problem

Q: What causes the "vanishing gradient" problem during backpropagation through time, and what issue does it create?

Solution :

The **vanishing gradient problem** occurs in RNNs when gradients become extremely small during Backpropagation Through Time (BPTT). This mainly happens because the gradient is repeatedly multiplied by the same recurrent weight matrix and derivatives of the activation function across many time steps. When these values are smaller than 1, repeated multiplication can make the gradient decrease exponentially toward zero.

Example:

Suppose the gradient is multiplied by **0.5** at every time step:

Step 1: 0.5
Step 2: $0.5 \times 0.5 = 0.25$
Step 3: $0.25 \times 0.5 = 0.125$
Step 4: $0.125 \times 0.5 = 0.0625$
Step 5: $0.0625 \times 0.5 = 0.03125$

As the number of time steps increases, the gradient becomes closer and closer to zero.

Effect on Training:

When the gradient becomes very small, the corresponding weights receive extremely small updates. As a result, the RNN has difficulty learning **long-term dependencies**, meaning it struggles to use information from much earlier time steps when making predictions.

For example, in the sentence:

“I grew up in Bangladesh, and I speak fluent Bengali.”

The RNN may have difficulty retaining the important information **“Bangladesh”** when it reaches the later part of a long sequence.

A common solution is to use architectures such as **LSTM (Long Short-Term Memory)** and **GRU (Gated Recurrent Unit)**, which are designed to better preserve information and gradients over long sequences.

Question 5: Sequential Information Flow

Q: In forward propagation, why are inputs fed sequentially from oldest to newest data points?

Solution :

In an RNN, inputs are fed **sequentially from the oldest to the newest data points** because the RNN uses the hidden state from the previous time step to process the current input. This allows information from earlier data points to be carried forward and influence the processing of later data points.

The sequence is processed as:

$x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow x_3 \rightarrow h_3 \rightarrow \ldots \rightarrow x_{\square} \rightarrow h_{\square}$

Here, $\mathbf{h_{\square-1}}$ **is used to calculate** $\mathbf{h_{\square}}$, so the RNN needs to process the previous input before processing the current input.

Example:

Consider the sentence:

“I do not like this movie.”

The RNN processes the words in order:

“I” → “do” → “not” → “like” → “this” → “movie”

When processing **“like”**, the hidden state already contains information from **“I”, “do”, and “not”**. This helps the RNN understand the context of the current word.

Therefore, processing data from **oldest to newest** preserves the temporal or sequential relationship between the data points and allows previous information to influence future predictions.

Note: The chronological order is important when the order of the data carries meaning, such as text, speech, stock prices, or sensor readings.